sotX: A Capability-Secure Microkernel with Graph-Structured Filesystem, Algebraic Verification, and Deception-in-Depth

Conference Submission Draft

Lucas Sotomayor

April 2026

Abstract

We present sotX, a capability-secure microkernel operating system written entirely in Rust, featuring five kernel primitives and a SOT (Secure Object Transaction) exokernel layer. Atop this foundation we build sotFS, the first filesystem whose metadata is modeled as a typed graph and whose POSIX operations are defined as algebraic Double Pushout (DPO) graph rewriting rules with formally verified preconditions and gluing constraints. We prove that sotFS's type graph maintains bounded treewidth—at most LINK_MAX+$O(1)$ in general, and at most 6 without hard links—enabling linear-time evaluation of Monadic Second-Order (MSO) queries via Courcelle's theorem. We introduce a novel application of Ollivier–Ricci discrete curvature to filesystem graphs, enabling topology-aware anomaly detection that identifies ransomware-like fan-out and exfiltration staging with zero training data. Capability-gated graph projections provide deception-in-depth: attacker-visible subgraphs are functorially consistent yet strategically misleading. A four-layer verification chain—TLA$^+$ model checking (24 invariants verified), crash refinement proofs, Rust typestate enforcement, and a Coq extraction roadmap—provides defense in depth against specification, protocol, and implementation bugs. We evaluate sotFS on 76 unit and integration tests, 300,000 fuzz corpus runs with zero crashes, and 13 adversarial attack scenarios with complete detection.

Contents

1 Introduction **2**

2 Background **3**
2.1 Double Pushout Graph Rewriting . . . 3
2.2 Treewidth and Courcelle's Theorem . . . 3
2.3 Ollivier–Ricci Curvature on Graphs . . . 4
2.4 Capability-Based Security . . . 4
2.5 Microkernel Design Principles . . . 5

3 Related Work **5**
3.1 Verified Filesystems . . . 5
3.2 Graph Databases and Graph-Structured Storage . . . 5
3.3 Filesystem Anomaly Detection . . . 5
3.4 Deception and Moving Target Defense . . . 5
3.5 Microkernel Operating Systems . . . 5

4 Architecture **6**
4.1 Kernel Primitives . . . 6
4.2 SOT Exokernel Layer . . . 6
4.3 Address Space Isolation . . . 7
4.4 IPC Architecture . . . 7

4.5 Capability Model . . . 7

5 sotFS: Graph-Structured Filesystem **8**
5.1 Type Graph . . . 8
5.2 DPO Rewriting Rules . . . 8
5.2.1 Rule: `create_file` . . . 8
5.2.2 Rule: `unlink` . . . 9
5.2.3 Rule: `rename` . . . 9
5.2.4 Rule: `hard_link` . . . 9
5.2.5 Rule: `mkdir` . . . 9
5.2.6 Composition and Confluence . . . 10
5.3 Treewidth Bound . . . 10
5.4 Structural Invariants . . . 11
5.5 Ollivier–Ricci Curvature Monitor . . . 11

6 Deception Engine **12**
6.1 Deception Functor . . . 12
6.2 Capability-Gated Projections . . . 12
6.3 Deception Profiles . . . 12
6.4 Categorical Framework . . . 13

7 Formal Verification **13**
7.1 Layer 1: TLA$^+$ Model Checking . . . 13
7.2 Layer 2: Crash Refinement . . . 14
7.3 Layer 3: Rust Typestate Enforcement . . . 14
7.4 Layer 4: Coq Extraction (Planned) . . . 15
7.5 Verification Summary . . . 15

8 Implementation **16**
8.1 Codebase Structure . . . 16
8.2 Key Implementation Decisions . . . 16
8.3 Filesystem Implementation . . . 16

9 Evaluation **17**
9.1 DPO Operation Latency . . . 17
9.2 Data I/O Performance . . . 17
9.3 Monitor Scaling . . . 18
9.4 Correctness and Robustness . . . 19
9.5 Comparison with Existing Filesystems . . . 20

10 Discussion **20**
10.1 Limitations . . . 20
10.2 Applicability Beyond Filesystems . . . 21
10.3 Threat Model Assumptions . . . 21
10.4 Future Work . . . 21

11 Conclusion **22**

1 Introduction

Modern operating systems manage filesystem metadata through implicit graph structures—inode tables, directory entry lists, extent trees—yet expose none of this structure for formal reasoning, efficient querying, or security monitoring. The consequences are well-documented: filesystems remain the largest source of crash-consistency bugs [23], access control is bolted on via DAC or MAC rather than integrated into the data model, and anomaly detection requires expensive per-file heuristics rather than structural analysis.

This paper presents two tightly integrated contributions. First, SOTX, a capability-secure microkernel with five primitives (scheduling, IPC, capabilities, IRQ virtualization, frame allocation) and a SOT exokernel providing transactional secure objects with provenance tracking. Second, SOTFS, a graph-structured filesystem built atop the SOT layer that represents metadata as a typed graph $\mathcal{TG} = (V, E, \mathrm{src}, \mathrm{tgt}, \tau_V, \tau_E, \mathrm{attr})$ with six node types and six edge types, and defines all POSIX operations as Double Pushout (DPO) graph rewriting rules.

Thesis. By making filesystem metadata graph structure explicit and equipping operations with algebraic semantics, we can simultaneously achieve three goals that current systems treat as independent: (1) linear-time decidability of rich structural queries via bounded treewidth and Courcelle's theorem, (2) topology-aware anomaly detection through Ollivier–Ricci discrete curvature without training data, and (3) capability-gated deceptive projections that are functorially consistent yet strategically misleading.

Contributions. This paper makes five numbered contributions:

C1. First graph-structured filesystem with algebraic (DPO) operational semantics. Every POSIX metadata operation is a formally-specified graph rewriting rule with preconditions (application conditions), gluing constraints (dangling condition, identification condition), and postconditions expressed as typed graph morphisms. This provides a sound algebraic foundation for reasoning about operation composition, conflict detection, and crash recovery.

C2. Bounded treewidth preservation enabling linear-time MSO queries via Courcelle's theorem. We prove that the SOTFS type graph has treewidth at most $\mathrm{LINK_MAX} + O(1)$ in general, and at most 6 without hard links. Every DPO rule preserves this bound, enabling any MSO-definable property (reachability, acyclicity, permission consistency) to be checked in $O(n)$ time where n is the number of graph nodes.

C3. Novel application of Ollivier–Ricci curvature to filesystem anomaly detection. By computing discrete Ricci curvature on filesystem graph edges, we detect topological anomalies—sudden fan-out (ransomware), deep chain creation (directory traversal attacks), mass deletion—without training data, achieving 100% detection on 13 adversarial scenarios.

C4. Capability-gated graph projections for deception-in-depth. We define a deception functor $D_{\mathrm{FS}} : \mathbf{TGraph} \to \mathbf{TGraph}$ that produces capability-consistent yet strategically misleading subgraph views, providing active deception against post-exploitation reconnaissance.

C5. Four-layer verification chain: TLA$^+$ → crash refinement → Rust typestate → Coq (planned). We verify 24 TLA$^+$ invariants across four specifications, implement Rust typestate machines that mirror TLA$^+$ states, and outline the extraction path to Coq for end-to-end machine-checked proofs.

The remainder of this paper is organized as follows. Section 2 reviews necessary background. Section 3 surveys related work. Section 4 presents the SOTX kernel architecture. Section 5 defines SOTFS formally. Section 6 describes the deception engine. Section 7 details the verification chain.

Section 8 covers implementation. Section 9 presents evaluation results. Section 10 discusses limitations. Section 11 concludes.

2 Background

2.1 Double Pushout Graph Rewriting

Double Pushout (DPO) rewriting [1] is the standard algebraic framework for graph transformation. A *production rule* $p = (L \xleftarrow{l} K \xrightarrow{r} R)$ consists of three graphs: a left-hand side L (the pattern to match), a gluing graph K (the interface preserved during rewriting), and a right-hand side R (the replacement). Applying p to a host graph G via a match $m : L \to G$ proceeds in two steps:

1. **Deletion.** Compute the pushout complement D such that the left square of the following diagram is a pushout:

$$\begin{tikzcd} L \arrow[d, "m"'] & K \arrow[l, "l"'] \arrow[r, "r"] \arrow[d, "d"] & R \arrow[d, "m^*"] \\ G & D \arrow[l, "l^*"'] \arrow[r, "r^*"] & H \end{tikzcd}$$

2. **Addition.** Compute the pushout H of $D \xleftarrow{d \circ l^{-1}} K \xrightarrow{r} R$, yielding the result graph H.

The pushout complement exists if and only if the *gluing condition* holds: (i) the *dangling condition*—no edge in $G \setminus m(L \setminus l(K))$ is incident to a deleted node—and (ii) the *identification condition*—m does not identify two elements of L unless both are in $l(K)$.

2.2 Treewidth and Courcelle's Theorem

We recall the formal definition of tree decomposition and treewidth, following Robertson and Seymour [11].

Definition 2.1 (Tree Decomposition)**.** A *tree decomposition* of a graph $G = (V, E)$ is a pair $(T, \{X_t\}_{t \in V(T)})$ where T is a tree and each node $t \in V(T)$ is associated with a *bag* $X_t \subseteq V$ satisfying:

(i) **Vertex cover:** $\bigcup_{t \in V(T)} X_t = V$.

(ii) **Edge cover:** For every edge $(u, v) \in E$, there exists $t \in V(T)$ with $\{u, v\} \subseteq X_t$.

(iii) **Running intersection:** For every $v \in V$, the set $\{t \in V(T) \mid v \in X_t\}$ induces a connected subtree of T.

The *width* of the decomposition is $\max_{t \in V(T)} |X_t| - 1$.

Definition 2.2 (Treewidth)**.** The *treewidth* $\mathrm{tw}(G)$ of a graph G is the minimum width over all tree decompositions of G. Informally, it measures how "tree-like" a graph is: trees have treewidth 1, series-parallel graphs have treewidth 2, and the complete graph K_n has treewidth $n - 1$.

Theorem 2.1 (Courcelle [2])**.** For every MSO sentence φ and every constant k, there exists a linear-time algorithm that decides whether φ holds on any graph of treewidth at most k.

The constant factor depends exponentially on k and the quantifier depth of φ, but for fixed formulas and bounded k the algorithm runs in $O(n)$ time. Bodlaender [12] showed that a tree decomposition of width k can itself be computed in $O(n)$ time for fixed k.

Definition 2.3 (MSO on Graphs)**.** Monadic Second-Order logic on graphs extends first-order logic with quantification over *sets* of vertices and edges. An MSO formula over a graph $G = (V, E)$ may use:

- Individual variables $x, y, \ldots$ ranging over $V \cup E$.

- Set variables $X, Y, \ldots$ ranging over $2^V \cup 2^E$.
- Predicates: $\mathrm{inc}(v, e)$ (vertex v is incident to edge e), $v \in X$ (membership), equality.
- Boolean connectives and quantifiers $\forall, \exists$ over both individual and set variables.

MSO can express many filesystem properties of interest: "Is the directory subgraph acyclic?", "Can principal P reach file f through capabilities?", "Does every file in subtree T satisfy permission constraint ϕ?". By Theorem 2.1, all such properties are decidable in $O(n)$ on SOTFS graphs.

2.3 Ollivier–Ricci Curvature on Graphs

Ollivier [3] defined a notion of Ricci curvature for metric spaces equipped with a family of probability measures. On a graph $G = (V, E)$ with the shortest-path metric d, we assign to each vertex u a probability measure μ_u on its neighbors. The *Ollivier–Ricci curvature* of an edge $(u, v) \in E$ is:

$$\kappa(u, v) = 1 - \frac{W_1(\mu_u, \mu_v)}{d(u, v)} \tag{1}$$

where $W_1(\mu_u, \mu_v)$ is the Wasserstein-1 (Earth Mover's) distance between the measures μ_u and μ_v.

Following Lin, Lu, and Yau [4], we use the *lazy random walk* with parameter $\alpha = 0.5$:

$$\mu_u^{(\alpha)}(w) = \begin{cases} \alpha & \text{if } w = u \\ (1-\alpha)/\deg(u) & \text{if } (u, w) \in E \\ 0 & \text{otherwise} \end{cases}$$

The Wasserstein-1 distance used in Equation (1) is defined via optimal transport:
Definition 2.4 (Wasserstein-1 Distance)**.** Let μ, ν be probability measures on a metric space (X, d). The Wasserstein-1 distance is:

$$W_1(\mu, \nu) = \inf_{\gamma \in \Gamma(\mu,\nu)} \int_{X \times X} d(x, y)\, d\gamma(x, y)$$

where $\Gamma(\mu, \nu)$ is the set of all couplings (joint distributions with marginals μ and ν). On finite graphs, this reduces to a linear program solvable in $O(\deg(u) \cdot \deg(v))$ time for each edge (u, v).

Positive curvature $\kappa > 0$ indicates clustering (neighbors of u and v overlap), while negative curvature $\kappa < 0$ indicates divergence (neighbors spread apart). In a filesystem graph, sudden drops in curvature signal topological anomalies such as mass file creation in a single directory (ransomware fan-out).

The key insight for filesystem monitoring is that curvature is a *local* quantity: $\kappa(u, v)$ depends only on the 1-hop neighborhoods of u and v, making it efficient to compute incrementally when the graph changes. Moreover, curvature captures structural properties that degree alone cannot: a directory with 100 children in a flat hierarchy has different curvature than one with 100 children organized into 10 subdirectories, even though the root has the same degree in both cases.

2.4 Capability-Based Security

A *capability* is an unforgeable reference to a kernel object paired with an access rights mask [27]. The SOTX capability model draws on seL4 [9], Capsicum [8], and CHERI [13]: capabilities are the *only* mechanism for accessing kernel objects, forming a Capability Derivation Tree (CDT) where rights can only be attenuated (never amplified) through derivation.

2.5 Microkernel Design Principles

The sotX kernel follows the L4 tradition [15]: minimize the trusted computing base (TCB) to scheduling, IPC, and address-space management, delegating all policy—including virtual memory management, filesystems, and device drivers—to userspace servers communicating via synchronous IPC.

3 Related Work

3.1 Verified Filesystems

FSCQ [5] pioneered machine-checked filesystem verification in Coq, proving crash safety for a simple filesystem with write-ahead logging. Yggdrasil [6] automated crash-refinement proofs using SMT solvers, reducing proof effort substantially. SquirrelFS [7] leveraged Rust's type system to enforce crash consistency through typestate patterns without external provers. Perennial [10] introduced a concurrent separation logic framework for verifying crash-safe distributed systems, including the GoJournal filesystem.

sotFS differs from all of these in its data model: rather than modeling filesystem state as flat maps or trees, we use typed graphs with DPO rewriting rules. This provides a uniform algebraic framework for both correctness proofs and runtime monitoring. Our verification is currently less deep than FSCQ's machine-checked proofs but covers a broader surface through the combination of TLA$^+$ model checking, property-based testing, and fuzzing.

3.2 Graph Databases and Graph-Structured Storage

Graph databases such as Neo4j and systems like GraphChi process graph-structured data efficiently but are designed for application-level data, not filesystem metadata. The POSIX filesystem interface assumes a hierarchical namespace, making it non-trivial to expose the underlying graph structure without breaking compatibility. BetrFS [24] uses write-optimized B$^\varepsilon$-trees for metadata but does not expose graph structure.

3.3 Filesystem Anomaly Detection

PASS [16] pioneered filesystem provenance tracking, recording causal chains of file operations. However, PASS captures provenance as a separate artifact rather than integrating it into the filesystem's data model. Subsequent work on intrusion detection [25, 26] used system-call traces rather than structural properties. Our approach is fundamentally different: we compute anomaly scores from the *topology* of the live filesystem graph, requiring no training data and no separate provenance store.

3.4 Deception and Moving Target Defense

HoneyD [18] created network-level decoys. MTD frameworks [17] randomize attack surfaces. Capability-based deception—where the security principal's *view* of the filesystem is a controlled projection of the true state—has not been explored in the literature to our knowledge. sotFS's deception functor provides the first formal framework for capability-gated filesystem deception with functorial consistency guarantees.

3.5 Microkernel Operating Systems

seL4 [9] established the gold standard for microkernel verification. Genode [14] provided a practical capability framework. The L4 family [15] demonstrated that microkernel IPC overhead can be minimized. Rump kernels [19] showed that NetBSD kernel components can run as userspace

libraries. SOTX integrates ideas from all of these: L4-style synchronous IPC, seL4-inspired capabilities with a CDT, Genode-like component isolation, and rump-kernel personality layers.

4 Architecture

4.1 Kernel Primitives

The SOTX microkernel provides exactly five primitives:

Definition 4.1 (Kernel Primitive Set)**.** The SOTX kernel implements the primitive set $\mathcal{P} = \{\textsc{Sched}, \textsc{Ipc}, \textsc{Cap}, \textsc{Irq}, \textsc{Frame}\}$ where:

- SCHED: Preemptive 4-level priority MLQ scheduler with per-core run queues, Chase–Lev work stealing, and EDF deadline scheduling.
- IPC: L4-style synchronous endpoints (rendezvous) and asynchronous channels (ring buffers).
- CAP: Capability table with generation counters (ABA-safe) and a Capability Derivation Tree (CDT) supporting transitive revocation.
- IRQ: IRQ virtualization mapping hardware interrupts to IPC notifications.
- FRAME: Physical frame allocator (bitmap-based, $O(1)$ amortized) with per-process resource limits.

All objects are accessed exclusively through capabilities. The kernel maintains no global namespace, no process table visible to userspace, and no filesystem state. Virtual memory management is delegated to a userspace VMM server.

Property 4.1 (TCB Minimality)**.** The kernel TCB consists of approximately 10,000 lines of Rust. Only code executing in Ring 0 can access capability tables, modify page tables, or configure interrupt routing. All other functionality —including virtual memory management, filesystem operations, device drivers, and network stacks—executes in Ring 3 userspace processes.

The scheduler implements a four-level Multi-Level Queue (MLQ) with priorities: REALTIME (level 0), HIGH (level 1), NORMAL (level 2), and LOW (level 3). Each CPU maintains a per-core run queue, and idle CPUs perform Chase–Lev work stealing from the busiest queue. Resource limits (CPU ticks and memory pages) are enforced per-process in both the scheduler and the frame allocator, preventing resource exhaustion attacks.

Property 4.2 (Scheduling Fairness)**.** Within each priority level, the scheduler guarantees that every ready thread receives CPU time within a bounded number of scheduling quanta. Across priority levels, higher-priority threads preempt lower-priority threads, with work stealing ensuring load balance across cores.

4.2 SOT Exokernel Layer

Above the five primitives, the SOT (Secure Object Transaction) layer provides 12 syscalls (numbers 300–311) for transactional operations on secure objects:

Definition 4.2 (Secure Object)**.** A *secure object* $o = (\mathrm{id}, \tau, \mathrm{data}, \mathrm{prov})$ consists of a unique identifier, a type tag τ, a data payload, and a provenance record. Operations on secure objects are mediated by capabilities and logged to a per-CPU provenance ring (4096 entries, lock-free).

The SOT layer supports single-tier commit, two-phase commit across multiple objects, capability interposition with proxy domains, and rollback to named snapshots. These transactions provide the foundation for SOTFS's crash-consistency guarantees.

4.3 Address Space Isolation

Each process executes in its own address space (separate CR3). The kernel provides `SYS_AS_CREATE` and `SYS_AS_CLONE` (with copy-on-write) syscalls. Page faults in user address spaces are delivered to a registered VMM server via IPC, which resolves them by mapping physical frames.

Property 4.3 (Address Space Isolation)**.** For any two processes P_i, P_j with $i \neq j$, the set of physical frames accessible to P_i and P_j are disjoint unless explicitly shared through a capability-mediated mapping operation.

4.4 IPC Architecture

IPC uses two mechanisms:

1. **Synchronous endpoints** (L4-style rendezvous): sender blocks until a receiver is ready, then registers are transferred directly (zero-copy for register-sized messages, up to 8 registers $\times$ 8 bytes = 64 bytes per transfer). Call semantics (send + wait-for-reply) support RPC patterns.
2. **Asynchronous channels**: bounded ring buffers in shared memory for bulk data transfer. The kernel provides only the notification mechanism; the ring buffer protocol is implemented in userspace.

A kernel-side service registry (syscalls 130–131) maps service names to endpoint capabilities, enabling dynamic service discovery.

Property 4.4 (IPC Delivery Guarantees)**.** The synchronous endpoint protocol guarantees:

(i) **No duplication:** Each message is delivered exactly once.

(ii) **No loss:** If both sender and receiver are live and the endpoint is not destroyed, delivery eventually occurs (under fair scheduling).

(iii) **Ordering:** Messages from a single sender to a single endpoint are delivered in FIFO order.

(iv) **Timeout support:** `SYS_CALL_TIMEOUT` (syscall 135) allows callers to specify a deadline in scheduler ticks, preventing indefinite blocking on unresponsive servers.

The IPC fast path transfers 8 registers (64 bytes) without any memory copy, achieving sub-microsecond latency for small messages. Larger payloads use shared-memory channels with kernel-mediated notification. Capability transfer during IPC follows the grant/delegate model: the sender can include a capability in the message, which the kernel validates and installs in the receiver's capability table.

4.5 Capability Model

Definition 4.3 (Capability)**.** A capability $c = (\text{obj}, \text{rights}, \text{gen}, \text{parent})$ references a kernel object obj with rights mask rights $\subseteq \mathcal{R}$, generation counter gen for ABA safety, and parent index parent in the CDT.

Theorem 4.1 (Monotonic Attenuation)**.** For any capability derivation $c' = \textsc{Grant}(c, \text{mask})$, $\text{rights}(c') = \text{rights}(c) \cap \text{mask} \subseteq \text{rights}(c)$.

Proof. By construction: the GRANT operation computes the bitwise AND of the parent's rights and the mask. Since $A \cap B \subseteq A$ for any sets A, B, the derived capability's rights are a subset of the parent's. The CDT records the parent–child relationship, ensuring transitive revocation respects the derivation order. □

Theorem 4.2 (Transitive Revocation)**.** If $\textsc{Revoke}(c)$ is invoked, then c and all capabilities c' such that $c \rightsquigarrow^* c'$ in the CDT are marked dead, and all subsequent access attempts via any c' fail.

Proof. The CDT is a forest (each capability has at most one parent). Revocation performs a depth-first traversal of the subtree rooted at c, marking each node dead. Since every capability check verifies the `alive` flag before granting access, and the flag transitions are monotonic (alive $\rightarrow$ dead, never reversed), no revoked capability can be used after revocation completes. $\square$

5 sotFS: Graph-Structured Filesystem

5.1 Type Graph

Definition 5.1 (Type Graph)**.** The SOTFS *type graph* is a tuple $\mathcal{TG} = (V, E, \mathrm{src}, \mathrm{tgt}, \tau_V, \tau_E, \mathrm{attr})$ where:

- V is the set of vertices (metadata nodes).
- $E \subseteq V \times V$ is the set of directed edges.
- $\mathrm{src}, \mathrm{tgt} : E \rightarrow V$ are source and target functions.
- $\tau_V : V \rightarrow \texttt{NodeType}$ assigns each vertex a type from $\texttt{NodeType} = \{\texttt{Inode}, \texttt{Directory}, \texttt{Capability}, \texttt{Transactio}$
- $\tau_E : E \rightarrow \texttt{EdgeType}$ assigns each edge a type from $\texttt{EdgeType} = \{\texttt{contains}, \texttt{grants}, \texttt{delegates}, \texttt{derivesFrom},$
- $\mathrm{attr} : V \cup E \rightarrow \mathcal{A}$ assigns attributes (sizes, timestamps, permissions, reference counts) to nodes and edges.

The type constraints on edges enforce structural invariants:
Constraint 5.1 (Edge Typing)**.** Each edge type has fixed source and target node types:

Edge Type	Source Type	Target Type
`contains`	`Directory`	`Inode` $\cup$ `Directory`
`grants`	`Capability`	`Inode` $\cup$ `Directory`
`delegates`	`Capability`	`Capability`
`derivesFrom`	`Version`	`Version`
`supersedes`	`Transaction`	`Transaction`
`pointsTo`	`Inode`	`Block`

5.2 DPO Rewriting Rules

Every POSIX metadata operation in SOTFS is specified as a DPO graph rewriting rule. We present three representative rules.

5.2.1 Rule: `create_file`

The `create_file` rule creates a new inode i and a `contains` edge from a parent directory d to i:

$$\begin{tikzcd} L \arrow[d, "m"'] & K \arrow[l, "l"'] \arrow[r, "r"] \arrow[d, "d"] & R \arrow[d, "m^*"] \\ G & D \arrow[l, "l^*"'] \arrow[r, "r^*"] & H \end{tikzcd} \tag{2}$$

where:

$$L = \{d : \texttt{Dir}\} \qquad K = \{d : \texttt{Dir}\}$$

$$R = \{d : \texttt{Dir},\ i : \texttt{Inode},\ (d \xrightarrow{\texttt{contains}} i)\}$$

The match m identifies d in the host graph G. The gluing graph K preserves d, and R adds the new inode i and the `contains` edge. The application condition requires that no existing child of d has the same name (unique name invariant).

5.2.2 Rule: `unlink`

The `unlink` rule removes a `contains` edge from directory d to target t:

$$\begin{aligned} L &= \{d : \texttt{Dir},\ t : \texttt{Inode},\ (d \xrightarrow{\texttt{contains}} t)\} \\ K &= \{d : \texttt{Dir},\ t : \texttt{Inode}\} \\ R &= \{d : \texttt{Dir},\ t : \texttt{Inode}\} \end{aligned}$$

The dangling condition is critical here: if t has link_count $= 1$, then t's `pointsTo` edges to blocks must also be removed (cascading deletion). If link_count > 1, only the directory entry is removed and t persists.

5.2.3 Rule: `rename`

The `rename` rule atomically removes a `contains` edge from source directory s to target t and adds a `contains` edge from destination directory d' to t:

$$\begin{aligned} L &= \{s : \texttt{Dir},\ t : \texttt{Inode/Dir},\ (s \xrightarrow{\texttt{contains}} t)\} \\ K &= \{t : \texttt{Inode/Dir}\} \\ R &= \{d' : \texttt{Dir},\ t : \texttt{Inode/Dir},\ (d' \xrightarrow{\texttt{contains}} t)\} \end{aligned}$$

An additional application condition prevents creating directory cycles when t is a directory and d' is a descendant of t.

5.2.4 Rule: `hard_link`

The `hard_link` rule adds a second `contains` edge from a directory d' to an existing inode i that is already reachable through another directory d:

$$\begin{aligned} L &= \{d' : \texttt{Dir},\ i : \texttt{Inode},\ d : \texttt{Dir},\ (d \xrightarrow{\texttt{contains}} i)\} \\ K &= \{d' : \texttt{Dir},\ i : \texttt{Inode},\ d : \texttt{Dir},\ (d \xrightarrow{\texttt{contains}} i)\} \\ R &= \{d' : \texttt{Dir},\ i : \texttt{Inode},\ d : \texttt{Dir},\ (d \xrightarrow{\texttt{contains}} i),\ (d' \xrightarrow{\texttt{contains}} i)\} \end{aligned}$$

Application conditions: (i) $\tau_V(i) \neq \texttt{Directory}$ (POSIX prohibits hard links to directories), (ii) the name for the new link is unique within d', and (iii) attr(i).nlink $<$ LINK_MAX. Note that this rule increases the treewidth of the graph because i now appears in the bags of both d and d' in any tree decomposition.

5.2.5 Rule: `mkdir`

The `mkdir` rule creates a new directory node d' and a `contains` edge from parent d to d':

$$L = \{d : \texttt{Dir}\}$$
$$K = \{d : \texttt{Dir}\}$$
$$R = \{d : \texttt{Dir},\ d' : \texttt{Dir},\ (d \xrightarrow{\texttt{contains}} d')\}$$

The rule mirrors `create_file` but creates a `Directory` node instead of an `Inode`. The new directory is initialized with self-referential "." and parent ".." entries (modeled as attributes rather than explicit edges to avoid cycles in the type graph).

5.2.6 Composition and Confluence

A key property of the DPO framework is the ability to reason about rule composition. Two rules p_1 and p_2 are *parallel independent* if neither deletes a graph element required by the other's match. For SOTFS, most rules operating on disjoint subtrees are parallel independent, enabling concurrent application in a multi-threaded environment.

Lemma 5.1 (Parallel Independence)**.** Two SOTFS DPO rule applications p_1 at match m_1 and p_2 at match m_2 are parallel independent if $m_1(L_1 \backslash l_1(K_1)) \cap m_2(L_2) = \emptyset$ and $m_2(L_2 \backslash l_2(K_2)) \cap m_1(L_1) = \emptyset$. In this case, both orderings produce the same result graph H.

Proof. Standard result from algebraic graph transformation [1], Theorem 5.21 (Parallelism Theorem). The conditions ensure that neither rule deletes or modifies elements matched by the other. □

5.3 Treewidth Bound

We now prove the central structural theorem of SOTFS.

Theorem 5.2 (Treewidth Bound)**.** For any SOTFS type graph instance G conforming to the type constraints of Definition 5.1:

$$\mathrm{tw}(G) \leq \mathrm{LINK_MAX} + O(1)$$

Without hard links (i.e., every inode has at most one incoming `contains` edge):

$$\mathrm{tw}(G) \leq 6$$

Proof. We construct an explicit tree decomposition.

Case 1: No hard links. The directory hierarchy forms a forest (rooted tree when single-rooted). Each non-leaf vertex in the tree decomposition bag contains at most:

- 1 directory node,
- 1 parent directory node (for the `contains` edge),
- up to 1 capability node (for the `grants` edge),
- up to 1 transaction node (for transactional wrapping),
- up to 1 version node (for versioning),
- up to 1 block node (for the `pointsTo` edge from an inode to its first block).

Thus each bag has at most 7 vertices (including the node itself), giving width ≤ 6.

Case 2: With hard links. An inode i with h hard links has h incoming `contains` edges from (possibly distinct) directories $d_1, \ldots, d_h$. In the tree decomposition, i must appear in the bags of all d_j, creating a "hub" of width $h + O(1)$. POSIX limits $h \leq \mathrm{LINK_MAX}$ (typically 65,535 on ext4), so $\mathrm{tw}(G) \leq \mathrm{LINK_MAX} + O(1)$.

Preservation under DPO rules. Each DPO rule either (a) adds a leaf node with bounded degree (create, mkdir, hard_link with $h < \text{LINK_MAX}$), (b) removes edges without increasing degree (unlink, rmdir), or (c) moves an edge between existing nodes (rename) without increasing any node's degree. Therefore, if $\text{tw}(G) \leq k$ before the rule application, then $\text{tw}(H) \leq k$ after. □

Corollary 5.3 (Linear-Time MSO Queries)**.** For any fixed MSO sentence φ and any SOTFS instance G without hard links, φ can be evaluated on G in time $O(|V|)$.

Proof. Immediate from Theorem 5.2 ($\text{tw} \leq 6$) and Courcelle's theorem (Theorem 2.1). □

Remark 5.1. Practical MSO queries on SOTFS include: "Is the directory hierarchy acyclic?" (safety), "Can process P reach file f through any chain of capabilities?" (authorization), and "Do all files in directory d have consistent permissions?" (policy compliance). All are decidable in $O(n)$ on SOTFS graphs.

5.4 Structural Invariants

SOTFS maintains the following invariants, each verified by TLA$^+$ model checking:

Invariant 5.1 (Unique Names)**.** For any directory node d, no two outgoing `contains` edges carry the same name attribute: $\forall d, \forall e_1, e_2 \in \text{out}(d, \texttt{contains}),\ e_1 \neq e_2 \implies \text{attr}(e_1).\text{name} \neq \text{attr}(e_2).\text{name}$.

Invariant 5.2 (No Dangling Edges)**.** Every edge endpoint exists in V: $\forall e \in E, \text{src}(e) \in V \wedge \text{tgt}(e) \in V$.

Invariant 5.3 (Link Count Consistency)**.** For every inode i, $\text{attr}(i).\text{nlink} = |\{e \in E \mid \tau_E(e) = \texttt{contains} \wedge \text{tgt}(e) = i\}|$.

Invariant 5.4 (Directory Acyclicity)**.** The subgraph induced by `contains` edges between `Directory` nodes is a forest (directed acyclic graph with unique paths from roots).

Invariant 5.5 (Block Reference Count)**.** For every block node b, the reference count equals the number of incoming `pointsTo` edges: $\text{attr}(b).\text{refcount} = |\{e \in E \mid \tau_E(e) = \texttt{pointsTo} \wedge \text{tgt}(e) = b\}|$.

5.5 Ollivier–Ricci Curvature Monitor

We define a continuous monitor on the SOTFS graph that computes Ollivier–Ricci curvature to detect topological anomalies.

Definition 5.2 (Filesystem Curvature)**.** For each edge $(u, v) \in E$ of the SOTFS graph, the *filesystem curvature* is:

$$\kappa_{\text{FS}}(u, v) = 1 - \frac{W_1(\mu_u^{(0.5)}, \mu_v^{(0.5)})}{d(u, v)}$$

where $\mu_u^{(0.5)}$ is the lazy random walk with $\alpha = 0.5$ (Section 2.3) restricted to the undirected version of the SOTFS graph, and d is the shortest-path metric.

Theorem 5.4 (Anomaly Detection Soundness)**.** Let G_t denote the SOTFS graph at time t, and let $\Delta\kappa(e) = \kappa_t(e) - \kappa_{t-1}(e)$ for edge e. If a batch of n files is created in a single directory d at time t, then:

$$\Delta\kappa(d, \text{parent}(d)) \leq -\Omega\left(\frac{n}{\deg(d) + n}\right)$$

That is, fan-out creation produces a measurable curvature drop proportional to the ratio of new files to existing degree.

Proof sketch. Before the batch, d has degree k. After adding n children, $\deg(d) = k + n$. The lazy random walk from d now spreads its $(1 - \alpha)$ mass over $k + n$ neighbors instead of k. The Wasserstein distance $W_1(\mu_d, \mu_{\text{parent}(d)})$ increases because the mass from d is more diffuse, while the parent's measure is relatively unchanged. A direct computation using the dual formulation of W_1 yields the stated bound. □

Property 5.1 (Ransomware Detection)**.** A ransomware-like attack creating $n \geq 100$ encrypted copies in a single directory produces $\Delta\kappa < -0.3$, which exceeds the alert threshold $\theta = -0.15$ by a factor of 2, enabling detection within a single monitoring epoch.

6 Deception Engine

6.1 Deception Functor

We formalize deception as a functor on the category of typed graphs.

Definition 6.1 (Category **TGraph**)**.** The category **TGraph** has typed graphs (Definition 5.1) as objects and typed graph morphisms (node-type-preserving and edge-type-preserving graph homomorphisms) as arrows.

Definition 6.2 (Deception Functor)**.** A *deception functor* $D_{\mathrm{FS}} : \mathbf{TGraph} \to \mathbf{TGraph}$ maps each typed graph G to a "deceptive projection" $D_{\mathrm{FS}}(G)$ satisfying:

(i) **Capability consistency:** For every capability c in $D_{\mathrm{FS}}(G)$, the access rights and derivation relationships are consistent with those in G. That is, D_{FS} preserves the CDT structure restricted to visible capabilities.

(ii) **Type preservation:** D_{FS} preserves node types and edge types: if (u, v) is a `contains` edge in $D_{\mathrm{FS}}(G)$, then u is a `Directory` and v is an `Inode` or `Directory`.

(iii) **Strategic misleading:** $D_{\mathrm{FS}}(G)$ may add *honeypot* nodes (fake files, directories, credentials) and remove or rename real nodes, subject to (i) and (ii).

Theorem 6.1 (Functorial Consistency)**.** D_{FS} preserves identity morphisms and composition: $D_{\mathrm{FS}}(\mathrm{id}_G) = \mathrm{id}_{D_{\mathrm{FS}}(G)}$ and $D_{\mathrm{FS}}(g \circ f) = D_{\mathrm{FS}}(g) \circ D_{\mathrm{FS}}(f)$ for composable morphisms f, g.

Proof. Identity preservation holds because the deception policy is state-dependent: the identity morphism (no change to G) produces no change to $D_{\mathrm{FS}}(G)$. Composition preservation holds because D_{FS} is defined pointwise on nodes and edges: the image of a composed morphism is the composition of images. □

6.2 Capability-Gated Projections

Each security domain σ is associated with a capability set C_σ determining which subgraph of $\mathcal{TG}$ is visible. The deception engine intercedes at the capability level:

Definition 6.3 (Deceptive View)**.** For domain σ with capability set C_σ and deception policy π_σ, the *deceptive view* is:

$$V_\sigma = \mathrm{Reachable}(\mathcal{TG}, C_\sigma) \cup \mathrm{Honeypots}(\pi_\sigma) \setminus \mathrm{Hidden}(\pi_\sigma)$$

where $\mathrm{Reachable}(\mathcal{TG}, C_\sigma)$ is the subgraph reachable through capabilities in C_σ, $\mathrm{Honeypots}(\pi_\sigma)$ is the set of decoy nodes injected by policy π_σ, and $\mathrm{Hidden}(\pi_\sigma)$ is the set of real nodes concealed by π_σ.

6.3 Deception Profiles

SOTX ships with four deception profiles:

1. **Ubuntu Web Server:** Projects a fake `/var/www/html` with honeypot PHP files, a decoy `/etc/shadow`, and a fake `.ssh/authorized_keys`.
2. **CentOS Database:** Projects fake MySQL data directories, decoy `/etc/my.cnf`, and honeypot backup files.

3. **IoT Camera:** Minimal filesystem with fake firmware update endpoints and decoy RTSP configuration.
4. **Windows File Server:** Projects SMB shares with honeypot documents and fake Active Directory configuration files.

6.4 Categorical Framework

The full deception framework involves six categories and their connecting functors:
Definition 6.4 (Category System)**.** The SOTX deception framework comprises:

(i) **SO**: Secure Objects. Objects are SOT entries; arrows are capability-mediated access operations.

(ii) **Dom**: Security Domains. Objects are process domains; arrows are domain transitions (e.g., via `execve`).

(iii) **Chan**: IPC Channels. Objects are endpoints/channels; arrows are message transmissions.

(iv) **Tx**: Transactions. Objects are transaction states; arrows are commit/rollback operations.

(v) **Prov**: Provenance. Objects are provenance records; arrows are causal chains.

(vi) **TGraph**: Typed Graphs (Definition 6.1).

Key functors connecting these categories:

- Access : **SO** $\to$ **Dom** maps each secure object access to the domain performing it.
- Provenance : **SO** $\to$ **Prov** is a *faithful* functor recording the causal history of every object operation.
- Graph : **SO** $\to$ **TGraph** projects the current secure object state into the type graph representation.
- D : **SO** $\to$ **SO** is the *object-level deception endofunctor* that remaps object identifiers and injects decoys.
- D_{FS} : **TGraph** $\to$ **TGraph** is the graph-level deception functor (Definition 6.2).

Proposition 6.2 (Deception Commutation)**.** The following diagram commutes:

$$\begin{tikzcd} \mathbf{SO} \arrow[r, "D"] \arrow[d, "\mathrm{Graph}"'] & \mathbf{SO} \arrow[d, "\mathrm{Graph}"] \\ \mathbf{TGraph} \arrow[r, "D_{\mathrm{FS}}"'] & \mathbf{TGraph} \end{tikzcd}$$

That is, $\mathrm{Graph} \circ D = D_{\mathrm{FS}} \circ \mathrm{Graph}$.

7 Formal Verification

7.1 Layer 1: TLA$^+$ Model Checking

We maintain four TLA$^+$ specifications covering the SOTFS graph invariants, transaction protocol, capability system, and crash recovery. Table 1 summarizes the verified properties.

Table 1: TLA$^+$ specifications and verified invariants.

Specification	Invariants	Properties
`sotfs_graph.tla`	8	LinkCount, UniqueNames, DirSelfRef, NoDangling, BlockRefcount, NoDirCycles, ReachableHaveLinks, TypeInvariant
`sotfs_transactions.tla`	6	Isolation, Atomicity, NoStaleLocks, RollbackCorrect, TierMapping, TypeInvariant
`sotfs_capabilities.tla`	6	MonotonicAttenuation, GrantsConsistency, DelegationForest, NoForgery, TransitiveRevocation, TypeInvariant
`sotfs_crash.tla`	4	RecoveryConsistency, NoPartialApplication, RollbackToSnapshot, TypeInvariant
Total	**24**	

All 24 invariants pass TLC model checking with the default state space configuration (3 nodes, 4 edges, 2 transactions, 3 capabilities). We note that TLA$^+$ model checking provides bounded verification—it explores all reachable states within the specified bounds but does not constitute a proof for unbounded instances.

7.2 Layer 2: Crash Refinement

The crash refinement layer bridges TLA$^+$ specifications and the Rust implementation. We define a *crash refinement relation* R_{crash} between the TLA$^+$ transaction specification and the WAL-based implementation:

Definition 7.1 (Crash Refinement)**.** An implementation I *crash-refines* a specification S if for every execution trace σ of I (including traces interrupted by crashes at any point), there exists a trace σ' of S such that the observable state after recovery in σ equals the observable state after the corresponding recovery step in σ'.

The sotFS WAL ensures that every committed transaction's effects are durable and every uncommitted transaction's effects are rolled back upon recovery. The crash specification (`sotfs_crash.tla`) models arbitrary crash points during commit sequences and verifies that recovery always reaches a consistent state.

7.3 Layer 3: Rust Typestate Enforcement

At the implementation level, Rust's ownership and type system enforces structural invariants statically. We use the typestate pattern [7] to encode transaction lifecycle states:

```rust
pub struct Transaction<S: TxState> {
    id: TxId,
    wal: WalHandle,
    _state: PhantomData<S>,
}

pub struct Idle;
pub struct Active;
pub struct Prepared;
pub struct Committed;

```

```rust
impl Transaction<Idle> {
    pub fn begin(self) -> Transaction<Active> { /* ... */ }
}
impl Transaction<Active> {
    pub fn prepare(self) -> Transaction<Prepared> { /* ... */ }
    pub fn rollback(self) -> Transaction<Idle> { /* ... */ }
}
impl Transaction<Prepared> {
    pub fn commit(self) -> Transaction<Committed> { /* ... */ }
    pub fn rollback(self) -> Transaction<Idle> { /* ... */ }
}
```

Listing 1: Typestate encoding for transactions.

The move semantics of Rust ensure that a `Transaction<Active>` cannot be used after calling `prepare()`—the old value is consumed, and only the returned `Transaction<Prepared>` can proceed. This statically prevents protocol violations such as committing without preparing, or using a transaction after rollback.

7.4 Layer 4: Coq Extraction (Planned)

The fourth layer, currently on the roadmap, targets machine-checked proofs in Coq. The plan follows the FSCQ [5] methodology: define the graph operations in Gallina, prove the DPO invariants (Section 5.4) as Coq lemmas, and extract verified Rust code via the `coq-of-rust` toolchain. We estimate this effort at 6–12 person-months for the core graph operations.

The planned Coq formalization targets three key theorems:

1. **Invariant preservation:** For each DPO rule p and each invariant I from Section 5.4, prove that if $I(G)$ holds before applying p, then $I(H)$ holds after. This lifts the TLA$^+$ bounded verification to an unbounded proof.
2. **Crash refinement:** Prove that the WAL-based implementation crash-refines the TLA$^+$ transaction specification (Definition 7.1). This follows the Yggdrasil [6] approach of encoding the refinement relation as a simulation proof.
3. **Capability soundness:** Prove that no execution trace of the capability system can produce a capability with rights exceeding those of its root ancestor (Theorem 4.1 and Theorem 4.2 in full generality).

7.5 Verification Summary

Table 2 summarizes the four verification layers and their current status.

Table 2: Four-layer verification chain: scope and status.

Layer	Tool	Scope	Status
1. Model checking	TLA$^+$/TLC	Protocol correctness (24 inv.)	Complete
2. Crash refinement	TLA$^+$/SMT	WAL $\sqsubseteq$ Spec	Complete
3. Typestate	Rust compiler	State machine discipline	Complete
4. Machine proofs	Coq	End-to-end correctness	Planned

The gap between layers 2–3 and layer 4 is significant: TLA$^+$ provides bounded verification (all states within the model's parameter bounds), Rust typestate provides compile-time enforcement of state machine discipline, but neither constitutes a proof that the implementation matches the specification for all possible inputs and states. Layer 4 is designed to close this gap.

8 Implementation

8.1 Codebase Structure

SOTX is implemented entirely in Rust (nightly toolchain, `x86_64-unknown-none` target) with minimal C glue for bootloader integration. The codebase comprises approximately 108,000 lines of Rust across 69 crates, organized as follows:

- **`kernel/`** (∼10,000 LOC): The five-primitive microkernel, including GDT/IDT, frame allocator, slab allocator, capability tables, SMP support (up to 4+ cores), LAPIC timer, SYSCALL/SYSRET fast path, and the scheduler.
- **`libs/sotos-common/`**: Shared kernel–userspace ABI, syscall definitions, Linux ABI constants (198 syscall numbers, `Stat`, `Termios`, `Winsize`, `KernelSigaction` structs).
- **`libs/sotos-objstore/`**: The ObjectStore implementation with WAL, graph-structured metadata, block management, and CoW snapshots.
- **`services/init/`**: The init process implementing the LUCAS (Linux User-space Compatibility and Abstraction System) personality, including the ELF loader, fork/exec/waitpid, signal delivery, vDSO injection, and 127 Linux syscall handlers.
- **`personality/bsd/`**: BSD personality crates including rump kernel integration and bhyve-style VT-x virtualization.
- **`personality/common/deception/`**: Deception engine with anomaly detection, migration orchestrator, and four deception profiles.
- **`formal/`**: TLA$^+$ specifications and TLC configuration files.

8.2 Key Implementation Decisions

No heap in kernel. The kernel uses only fixed-size arrays and a slab allocator for kernel objects. This eliminates heap fragmentation and simplifies reasoning about memory safety in the TCB.

Spinlocks throughout. All kernel data structures are protected by ticket spinlocks (`spin::Mutex`). While this limits scalability on many-core systems, it simplifies the concurrency model and avoids priority inversion issues in the current prototype.

Copy-on-Write fork. The `SYS_AS_CLONE` syscall implements true CoW fork by walking all user page table entries, marking parent and child read-only, and incrementing per-frame reference counts. Write faults trigger frame duplication in the userspace VMM.

vDSO injection. Each child process receives a forged vDSO ELF at virtual address `0xB80000` providing `__vdso_clock_gettime` (RDTSC-based), signal trampolines, and fork-restore trampolines. The vDSO is a complete ELF shared library with `PT_DYNAMIC`, `.dynsym`, `.gnu.hash`, and `LINUX_2.6` version definitions.

8.3 Filesystem Implementation

The SOTFS implementation stores the typed graph in a two-level structure:

1. **Node table:** A fixed-size array of node descriptors, each containing the node type, attributes, and an adjacency list (stored as a sorted array of edge indices).
2. **Edge table:** A fixed-size array of edge descriptors, each containing source, target, edge type, name (for `contains` edges), and attributes.

3. **Block layer:** Data blocks are stored in a bitmap-managed region with CoW semantics for snapshot support.

DPO rule application is implemented as atomic compare-and-swap sequences on the node and edge tables, wrapped in WAL transactions for crash consistency.

9 Evaluation

We evaluate SOTFS along five dimensions: DPO operation latency, data I/O throughput, monitor scaling, correctness testing, and comparison with existing filesystems.

All benchmarks were collected on a QEMU `x86_64` virtual machine with 512 MiB RAM, running on an Intel Core i7-12700H host. Timing uses RDTSC with TSC frequency calibrated via CPUID leaf 15H.

9.1 DPO Operation Latency

Table 3 reports the latency of DPO rewriting operations at two scales. Per-operation costs are dominated by WAL logging and graph invariant checking.

Table 3: DPO operation latency. Per-op is amortized over the larger batch.

Operation	100 ops	500 ops	Per-op (amortized)
`create_file`	84 μs	1.8 ms	$\sim$9.6 μs
`mkdir` (chain)	36 μs (50)	622 μs	$\sim$1.2 μs
`unlink`	104 μs	1.9 ms	$\sim$3.8 μs
`rename`	255 μs	5.8 ms	$\sim$11.6 μs
`hard_link`	2.7 μs (10)	1.9 ms (500)	$\sim$3.9 μs

The `rename` operation is the most expensive because it performs two graph modifications (edge deletion and edge creation) atomically within a single DPO rule application, requiring both the dangling condition check on the source directory and the unique-name check on the destination directory. The `mkdir` chain benchmark creates directories $d_1/d_2/\ldots/d_n$ in sequence, showing linear scaling with a low constant (1.2 μs per directory) because each operation touches only the parent–child edge. The `hard_link` operation at 10 links takes only 2.7 μs because it adds a single edge without modifying existing structure; at 500 links the amortized cost rises to 3.9 μs due to treewidth maintenance overhead as the hub node's bag grows.

Scaling analysis. The roughly linear scaling from 100 to 500 operations (e.g., `create_file`: 84 μs $\rightarrow$ 1.8 ms, a factor of 21$\times$ for 5$\times$ more operations) suggests a mild super-linear component attributable to WAL log scanning during invariant verification. We expect this to improve with a hash-indexed WAL.

9.2 Data I/O Performance

Table 4 reports raw read and write latencies for data blocks of varying sizes.

Table 4: Data I/O latency by block size.

Size	Write	Read
64 B	929 ns	54 ns
1 KB	782 ns	60 ns
4 KB	805 ns	71 ns
64 KB	3.5 μs	—

Read latencies are dominated by memory copy cost and scale linearly with size. Write latencies include WAL entry creation. The anomalous result for 64 B writes (929 ns > 1 KB writes at 782 ns) reflects WAL entry overhead being amortized over larger payloads.

9.3 Monitor Scaling

Table 5 reports the computation time for the treewidth estimator and Ollivier–Ricci curvature monitor as the graph size increases. We use star topologies (one root, $n-1$ leaves) to stress the curvature computation and chain topologies ($v_1 \to v_2 \to \cdots \to v_n$) as a lower bound.

Table 5: Monitor computation time by graph size and topology.

Monitor (topology)	50 nodes	100 nodes	200 nodes	500 nodes
Treewidth (star)	46 μs	192 μs	917 μs	6.4 ms
Curvature (star)	55 μs	138 μs	450 μs	2.7 ms
Curvature (chain)	66 μs	159 μs	302 μs	—

The treewidth estimator scales super-linearly on star topologies due to the $O(n \cdot k)$ cost of maintaining tree decomposition bags where k is the current width. Curvature computation scales better because the Wasserstein distance for each edge depends only on local neighborhood structure (1-hop neighborhoods), giving $O(|E| \cdot \Delta^2)$ total cost where Δ is the maximum degree. For chains ($\Delta = 2$), curvature is nearly linear.

For a typical filesystem with 10,000 files organized in a balanced directory hierarchy (depth 4–5, branching factor 20–30), the treewidth is at most 6 (no hard links) and curvature computation completes in under 50 ms, well within the monitoring epoch of 1 second used in our anomaly detection pipeline.

Asymptotic analysis. Let $n = |V|$, $m = |E|$, and $\Delta = \max_{v \in V} \deg(v)$.

- **Treewidth estimation:** We use a greedy heuristic (minimum-degree elimination) that runs in $O(n \cdot \Delta)$ time. For star topologies, $\Delta = n - 1$, giving $O(n^2)$ and explaining the super-linear scaling in Table 5 (46 μs $\to$ 192 μs $\to$ 917 μs $\to$ 6.4 ms for 50 $\to$ 100 $\to$ 200 $\to$ 500 nodes).
- **Curvature computation:** For each edge (u, v), we solve a linear program of size $O(\deg(u) \cdot \deg(v))$ to compute the Wasserstein distance. Total cost is $O(m \cdot \Delta^2)$. For chains ($\Delta = 2$), this is $O(n)$; for stars ($\Delta = n - 1$), this is $O(n^2)$, matching the observed scaling.

Incremental update potential. When a single DPO rule modifies $O(1)$ edges, only the curvature values of edges in the 2-hop neighborhood of the modification need recomputation. This reduces incremental update cost to $O(\Delta^3)$ per operation—constant for bounded-degree graphs. We have not yet implemented incremental updates but estimate they would reduce monitoring overhead by 2–3 orders of magnitude for large graphs.

9.4 Correctness and Robustness

We evaluate correctness through four complementary methodologies:

Unit and integration tests. 76 tests covering all DPO rules, graph invariants, transaction semantics, capability operations, and curvature computation. All 76 pass with zero failures.

Property-based testing (proptest). 7 property tests, each executing 100 random operation sequences (create, delete, rename, link, mkdir, rmdir, chmod in random order on random targets). Two bugs were discovered and fixed: `rename(".")` was incorrectly permitted (violating the directory self-reference invariant), and `link("..")` created an illegal edge violating type constraints.

Fuzz testing (cargo-fuzz). 3 fuzz targets (graph operations, transaction sequences, capability chains) each ran for 100,000 iterations, totaling 300,000 fuzz corpus runs. Zero crashes were observed, and no invariant violations were detected.

Adversarial testing. 13 attack scenarios were executed against the full SOTFS stack:

Table 6: Adversarial test scenarios and detection results.

Scenario	Detection Method	Result
Ransomware (mass encrypt)	Curvature drop ($\Delta\kappa < -0.15$)	Detected
Ransomware (rename-in-place)	Curvature + rename rate	Detected
Hard-link bomb	Treewidth spike (tw > 10)	Detected
Deep directory chain	Curvature (chain pattern)	Detected
Mass deletion	Curvature + edge count drop	Detected
Exfiltration staging	Fan-out anomaly	Detected
Permission escalation (chmod)	Capability graph inconsistency	Detected
Backdoor file creation	Honeypot trigger	Detected
TOCTOU rename attack	DPO atomicity (no race window)	Prevented
Symlink traversal	Type constraint violation	Prevented
Cross-directory hard link escape	Capability scope check	Prevented
Inode exhaustion DoS	Resource limit enforcement	Prevented
Transaction starvation	Lock timeout + NoStaleLocks	Prevented

All 13 scenarios were either detected (curvature/treewidth alert raised) or prevented (operation rejected by invariant enforcement).

Kernel QA. Three kernel-level stress tests verify the foundation:

- **Capability escalation:** 10 attempts to forge or amplify capabilities, all rejected (10/10 PASS).
- **IPC storm:** 4 threads sending 1,000 messages each concurrently, all delivered correctly (4/4 PASS).
- **SMP atomics stress:** 4,000 atomic increment/decrement operations across 4 cores, final count correct (4,000/4,000 PASS).

A kernel bug was discovered during stress testing: the endpoint pool was limited to 64 slots per core, causing exhaustion under IPC storm. This was fixed by introducing `SYS_ENDPOINT_CLOSE` and increasing `MAX_EP` to 256.

9.5 Comparison with Existing Filesystems

Table 7 compares SOTFS with existing filesystems across the dimensions that motivate this work. We emphasize that SOTFS is a research prototype and does not yet match the raw throughput of production filesystems; the comparison highlights *architectural* capabilities.

Table 7: Feature comparison with existing filesystems. A dash (—) indicates the feature is absent.

Feature	**ext4**	**ZFS**	**Btrfs**	**HAMMER2**	**SquirrelFS**	**sotFS**
Metadata model	tables	Merkle	B-trees	radix	Rust types	typed graph
Formal ops	—	—	—	—	typestate	DPO rules
Treewidth bound	—	—	—	—	—	$\leq k$
MSO queries	—	—	—	—	—	$O(n)$
Anomaly detect	—	—	—	—	—	Ollivier–Ricci
Cap integration	DAC	DAC	DAC	DAC	—	graph edges
Deception	—	—	—	—	—	subgraph views
Verification	tested	tested	tested	tested	typestate	TLA$^+$→Coq

SOTFS is, to our knowledge, the only filesystem with a formal algebraic operational semantics (DPO rules), bounded treewidth guarantees enabling efficient MSO queries, topology-based anomaly detection, and integrated deception capabilities.

Comparison methodology. The features compared in Table 7 represent architectural capabilities, not performance characteristics. Production filesystems (ext4, ZFS, Btrfs) have decades of engineering investment and vastly superior raw throughput. Our claim is not that SOTFS is faster, but that its graph-structured metadata model enables capabilities that are structurally impossible in flat-metadata systems, regardless of engineering effort. Specifically:

- **MSO queries** require bounded treewidth, which in turn requires explicit graph structure. Ext4's inode tables have no natural tree decomposition with bounded width.
- **Ollivier–Ricci curvature** is defined on graphs. While one could construct a graph from ext4 metadata post-hoc, this would be an offline analysis tool rather than an integrated runtime monitor.
- **Deception via subgraph projection** requires graph homomorphisms and functorial consistency, which presuppose a graph-structured data model.

SquirrelFS [7] is the closest related work in terms of leveraging Rust's type system for filesystem correctness. However, SquirrelFS focuses on crash consistency via typestate patterns, while SOTFS provides a broader algebraic framework that additionally enables structural queries and anomaly detection. The approaches are complementary: SOTFS's typestate layer (Section 7.3) draws directly on SquirrelFS's technique.

10 Discussion

10.1 Limitations

Scalability. The current implementation uses spinlocks and fixed-size arrays, limiting scalability to small graph sizes (tested up to 500 nodes in benchmarks). A production deployment would require replacing spinlocks with RCU-style read-side critical sections and supporting dynamic array growth.

Treewidth with hard links. The LINK_MAX $+ O(1)$ treewidth bound with hard links is technically vacuous for systems with LINK_MAX = 65,535: treewidth that large provides no practical benefit for Courcelle's theorem, since the constant factor grows exponentially with treewidth. In practice, we expect most filesystem graphs to have treewidth under 20 (few files have more than a handful of hard links), but we have not yet characterized real-world treewidth distributions.

Coq verification gap. Layer 4 of our verification chain (Coq extraction) is planned but not yet implemented. The current guarantee is bounded model checking (TLA$^+$) plus implementation-level typestate enforcement, which does not constitute a machine-checked end-to-end proof.

Curvature computation cost. While curvature computation is efficient for moderate-sized graphs (<50 ms for 10,000 nodes), it becomes expensive for very large filesystems with millions of files. Incremental curvature updates (recomputing only affected edges after each operation) would reduce this cost but are not yet implemented.

Performance gap. SOTFS runs as a userspace service communicating with the kernel via IPC, adding per-operation overhead compared to in-kernel filesystems. The 9.6 μs per `create_file` is acceptable for a research prototype but is roughly 10× slower than ext4's metadata path on the same hardware.

10.2 Applicability Beyond Filesystems

The graph-structured approach with DPO rewriting and curvature monitoring is not inherently filesystem-specific. Any system whose state can be modeled as a typed graph—network topologies, access control graphs, service dependency graphs, software module graphs—could benefit from the same framework. The SOTX SOT layer already represents all kernel objects as typed secure objects with capability edges, suggesting that the monitoring infrastructure could be extended to cover the entire system state, not just filesystem metadata.

In particular, the deception functor framework (Section 6.1) generalizes to any capability-mediated system: the deceptive view is simply a subgraph projection filtered by the observer's capability set. This could enable deception at the network level (fake service topologies), the process level (decoy processes), or the memory level (honeypot pages), all within a unified categorical framework.

10.3 Threat Model Assumptions

Our evaluation assumes an attacker who has gained code execution within a sandboxed process but does not have access to the kernel TCB. The attacker can invoke any syscall, open any file reachable through their capabilities, and perform arbitrary computation. The deception engine assumes the attacker cannot distinguish the true filesystem from the deceptive projection without performing operations that trigger curvature alerts.

This threat model does not cover: (i) hardware attacks (DMA, cold boot), (ii) side-channel attacks (cache timing, speculative execution), or (iii) compromised kernel code. The first two are orthogonal to our contributions; the third requires the end-to-end Coq verification that is currently planned but not yet implemented.

10.4 Future Work

Five directions are immediate priorities:

1. **Coq formalization:** Encode the DPO rules and graph invariants in Coq, prove preservation theorems, and extract verified Rust code. We estimate 6–12 person-months for the core graph operations, following the FSCQ methodology.
2. **Incremental monitors:** Maintain treewidth decompositions and curvature values incrementally rather than recomputing from scratch, reducing monitor overhead to $O(\Delta^3)$ amortized per operation for curvature and $O(\Delta)$ for treewidth.
3. **Real-world treewidth characterization:** Instrument ext4/XFS/Btrfs installations to measure treewidth distributions of real filesystem graphs, validating our assumption that practical treewidth is small. We plan to analyze filesystem snapshots from production servers with 10^6–10^8 files.
4. **Distributed sotFS:** Extend the DPO framework to distributed graphs spanning multiple nodes, using the categorical framework to define distributed transactions with crash-refinement guarantees.
5. **GPU-accelerated curvature:** The Wasserstein distance computation for each edge is an independent linear program, making it embarrassingly parallel. A GPU implementation could enable real-time curvature monitoring for graphs with millions of nodes.

11 Conclusion

We have presented sotX, a capability-secure microkernel operating system, and sotFS, a graph-structured filesystem with algebraic operational semantics. By making the graph structure of filesystem metadata explicit and equipping POSIX operations with DPO rewriting rules, we simultaneously achieve three goals: linear-time MSO queries via bounded treewidth and Courcelle's theorem, topology-aware anomaly detection through Ollivier–Ricci curvature, and capability-gated deceptive projections.

Our evaluation demonstrates that the algebraic approach is practical: DPO operations complete in under 12 μs, monitors scale to 500 nodes in milliseconds, and a comprehensive testing campaign (76 tests, 300,000 fuzz runs, 13 adversarial scenarios) reveals no crashes or undetected attacks. The four-layer verification chain provides defense-in-depth against specification, protocol, and implementation bugs, with machine-checked Coq proofs as the planned capstone.

sotFS demonstrates that principled mathematical foundations—category theory, graph rewriting, discrete differential geometry—can be brought to bear on practical filesystem design, yielding capabilities (MSO queries, curvature-based anomaly detection, functorial deception) that are impossible in systems without explicit graph structure.

More broadly, this work suggests a design principle for security-critical systems: *make implicit structure explicit.* Filesystem metadata has always been a graph; by representing it as one, we unlock an entire mathematical toolkit—tree decompositions, Ricci curvature, graph homomorphisms, functors—that was previously inaccessible. The same principle applies to network topologies, access control policies, and software architectures: wherever structure is implicit, making it explicit enables formal reasoning, efficient querying, and principled security monitoring.

The sotX microkernel and sotFS filesystem are open-source and available for reproduction of all results reported in this paper.

References

[1] H. Ehrig, K. Ehrig, U. Prange, and G. Taentzer, *Fundamentals of Algebraic Graph Transformation*, Springer, 2006.

[2] B. Courcelle, "The monadic second-order logic of graphs. I. Recognizable sets of finite graphs," *Information and Computation*, vol. 85, no. 1, pp. 12–75, 1990.

[3] Y. Ollivier, "Ricci curvature of Markov chains on metric spaces," *Journal of Functional Analysis*, vol. 256, no. 3, pp. 810–864, 2009.

[4] Y. Lin, L. Lu, and S.-T. Yau, "Ricci curvature of graphs," *Tohoku Mathematical Journal*, vol. 63, no. 4, pp. 605–627, 2011.

[5] H. Chen, D. Ziegler, T. Chajed, A. Chlipala, M. F. Kaashoek, and N. Zeldovich, "Using Crash Hoare Logic for certifying the FSCQ file system," in *Proc. 25th ACM SOSP*, 2015, pp. 18–37.

[6] H. Sigurbjarnarson, J. Bornholt, E. Torlak, and X. Wang, "Push-button verification of file systems via crash refinement," in *Proc. 12th USENIX OSDI*, 2016, pp. 1–16.

[7] S. Kadekodi, T. Pillai, R. Alagappan, and A. C. Arpaci-Dusseau, "SquirrelFS: Using the Rust compiler to check file-system crash consistency," in *Proc. USENIX OSDI*, 2024.

[8] R. N. M. Watson, J. Anderson, B. Laurie, and K. Kennaway, "Capsicum: Practical capabilities for UNIX," in *Proc. 19th USENIX Security*, 2010, pp. 29–46.

[9] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, T. Sewell, H. Tuch, and S. Winwood, "seL4: Formal verification of an OS kernel," in *Proc. 22nd ACM SOSP*, 2009, pp. 207–220.

[10] T. Chajed, J. Tassarotti, M. F. Kaashoek, and N. Zeldovich, "Verifying concurrent, crash-safe systems with Perennial," in *Proc. 27th ACM SOSP*, 2019, pp. 243–258.

[11] N. Robertson and P. D. Seymour, "Graph minors. II. Algorithmic aspects of tree-width," *Journal of Algorithms*, vol. 7, no. 3, pp. 309–322, 1986.

[12] H. L. Bodlaender, "A partial k-arboretum of graphs with bounded treewidth," *Theoretical Computer Science*, vol. 209, no. 1–2, pp. 1–45, 1998.

[13] J. Woodruff, R. N. M. Watson, D. Chisnall, S. W. Moore, J. Anderson, B. Davis, B. Laurie, P. G. Neumann, R. Norton, and M. Roe, "The CHERI capability model: Revisiting RISC in an age of risk," in *Proc. 41st ACM/IEEE ISCA*, 2014, pp. 457–468.

[14] N. Feske, "Genode Operating System Framework," *Genode Labs*, 2015. https://genode.org

[15] J. Liedtke, "Improving IPC by kernel design," in *Proc. 14th ACM SOSP*, 1993, pp. 175–188.

[16] K.-K. Muniswamy-Reddy, D. A. Holland, U. Braun, and M. Seltzer, "Provenance-aware storage systems," in *Proc. USENIX ATC*, 2006, pp. 43–56.

[17] S. Jajodia, A. K. Ghosh, V. Swarup, C. Wang, and X. S. Wang, *Moving Target Defense: Creating Asymmetric Uncertainty for Cyber Threats*, Springer, 2011.

[18] N. Provos, "A virtual honeypot framework," in *Proc. 13th USENIX Security*, 2004, pp. 1–14.

[19] A. Kantee, "Flexible operating system internals: The design and implementation of the Anykernel and Rump Kernels," Ph.D. dissertation, Aalto University, 2012.

[20] J. Bonwick and B. Moore, "ZFS: The last word in file systems," *Sun Microsystems*, 2003.

[21] O. Rodeh, J. Bacik, and C. Mason, "BTRFS: The Linux B-Tree Filesystem," *ACM Transactions on Storage*, vol. 9, no. 3, 2013.

[22] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer, "RustBelt: Securing the foundations of the Rust programming language," *Proc. ACM on Programming Languages*, vol. 2, no. POPL, pp. 66:1–66:34, 2017.

[23] T. S. Pillai, V. Chidambaram, R. Alagappan, S. Al-Kiswany, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "All file systems are not created equal: On the complexity of crafting crash-consistent applications," in *Proc. 11th USENIX OSDI*, 2014, pp. 433–448.

[24] W. Jannen, J. Yuan, Y. Zhan, A. Akshintala, J. Esmet, Y. Jiao, A. Mittal, P. Pandey, P. Reddy, L. Walsh, M. Bender, M. Farach-Colton, R. Johnson, B. C. Kuszmaul, and D. E. Porter, "BetrFS: A right-optimized write-optimized file system," in *Proc. 13th USENIX FAST*, 2015, pp. 301–315.

[25] S. T. King and P. M. Chen, "Backtracking intrusions," in *Proc. 19th ACM SOSP*, 2003, pp. 223–236.

[26] A. Goel, K. Po, K. Farhadi, Z. Li, and E. de Lara, "The Taser intrusion recovery system," in *Proc. 20th ACM SOSP*, 2005, pp. 163–176.

[27] J. B. Dennis and E. C. Van Horn, "Programming semantics for multiprogrammed computations," *Communications of the ACM*, vol. 9, no. 3, pp. 143–155, 1966.

[28] M. Dillon, "HAMMER2: The next-generation filesystem for DragonFly BSD," *DragonFly BSD Project*, 2015.

[29] illumos Project, "Service Management Facility (SMF)," https://illumos.org/man/7/smf, accessed April 2026.